Project-Low Level Design

on

Dockerized Marketing PHP Laravel Service

Course Name: DevOps

Institution Name:Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	CHARU RATHOD	EN23CS3T1007
2.	RUDRAKSH SHARMA	EN22CS301832
3.	PRATHAM DESHMUKH	EN22CS301741
4.	PIYUSH PATIDAR	EN22CS301705
5.	ROHAN JAIN	EN22CS301817
6.	PRIYANSH MEWADA	EN22CS301763

Group Name: D7

Project Number:DO-26

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-2026

Table of Contents

1. Introduction

The Dockerized Marketing PHP Laravel Service is a DevOps-focused project designed to containerize, deploy, and manage a PHP Laravel-based marketing web application using modern DevOps tools and practices. The system leverages Docker for containerization, Git for version control, Jenkins/GitHub Actions for CI/CD pipelines, and Kubernetes (K8s) for container orchestration, ensuring seamless deployment, scalability, and reliability of the marketing platform

1.1. Scope of the document

This document defines the Low-Level Design (LLD) of the Dockerized Marketing PHP Laravel Service.
It covers:
• Internal system architecture and container design
• Component-level design of Laravel service modules
• CI/CD pipeline flow using Jenkins/GitHub Actions
• Kubernetes deployment and orchestration design
• APIs and service structures The goal is to provide a detailed technical blueprint for developers and DevOps engineers to implement, deploy, and maintain the system..

1.2. Intended Audience

The intended audience for the Dockerized Marketing PHP Laravel Service includes DevOps engineers, backend developers, system architects, and project managers involved in deploying and maintaining the marketing platform. This system helps them by providing a structured containerized deployment plan based on the application stack and infrastructure requirements, reducing the effort of manually configuring servers and deployment pipelines. It is especially useful for teams who want an automated, scalable, and reliable deployment workflow, enabling them to ship features faster and manage their infrastructure more effectively.

1.3. System overview

The Dockerized Marketing PHP Laravel Service is a containerized DevOps system that:
• Accepts source code commits via Git (GitHub/GitLab)
• Triggers automated CI/CD pipelines using Jenkins or GitHub Actions
• Builds and pushes Docker images of the Laravel application
• Deploys containers to a Kubernetes cluster
• Uses multiple containerized services: Laravel App Container → PHP application runtime Nginx Container → Web server / reverse proxy MySQL Container → Database service Redis Container → Caching and queue management
• Ensures high availability and scalability via K8s autoscaling
• Produces a fully automated, reproducible deployment pipelin

2. System Design

2.1. Application Design

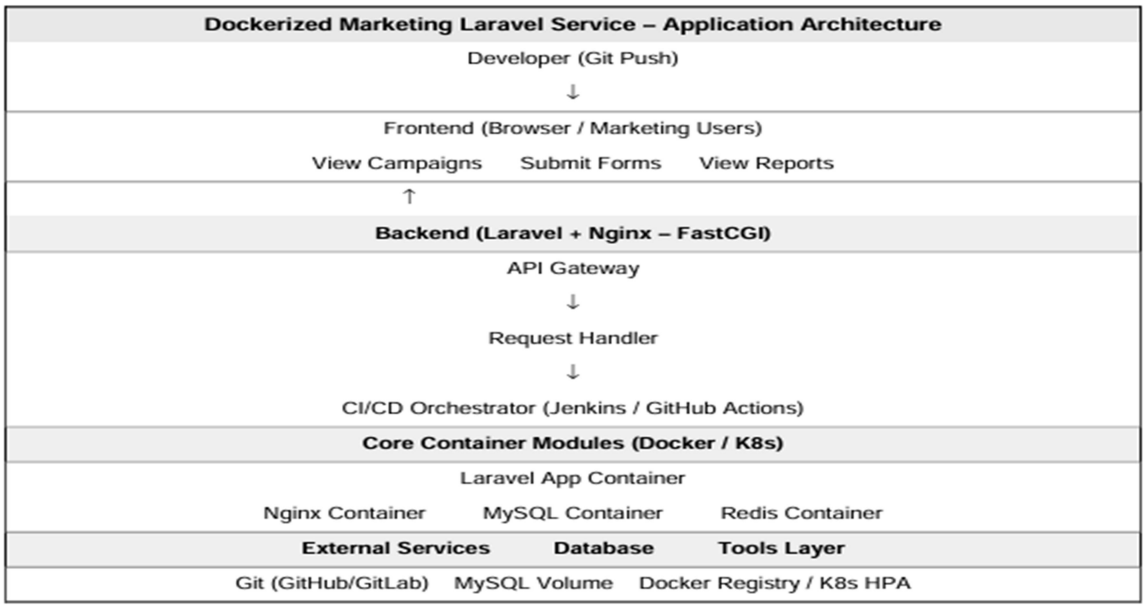
Architecture Type: Containerized Microservice Architecture with CI/CD Automation Layers:
• Source Control (Git) → GitHub/GitLab repository for Laravel codebase

• CI/CD Layer (Jenkins/Actions) → Automated build, test, and deploy pipelines • Container Layer (Docker) → Dockerized Laravel, Nginx, MySQL, Redis services

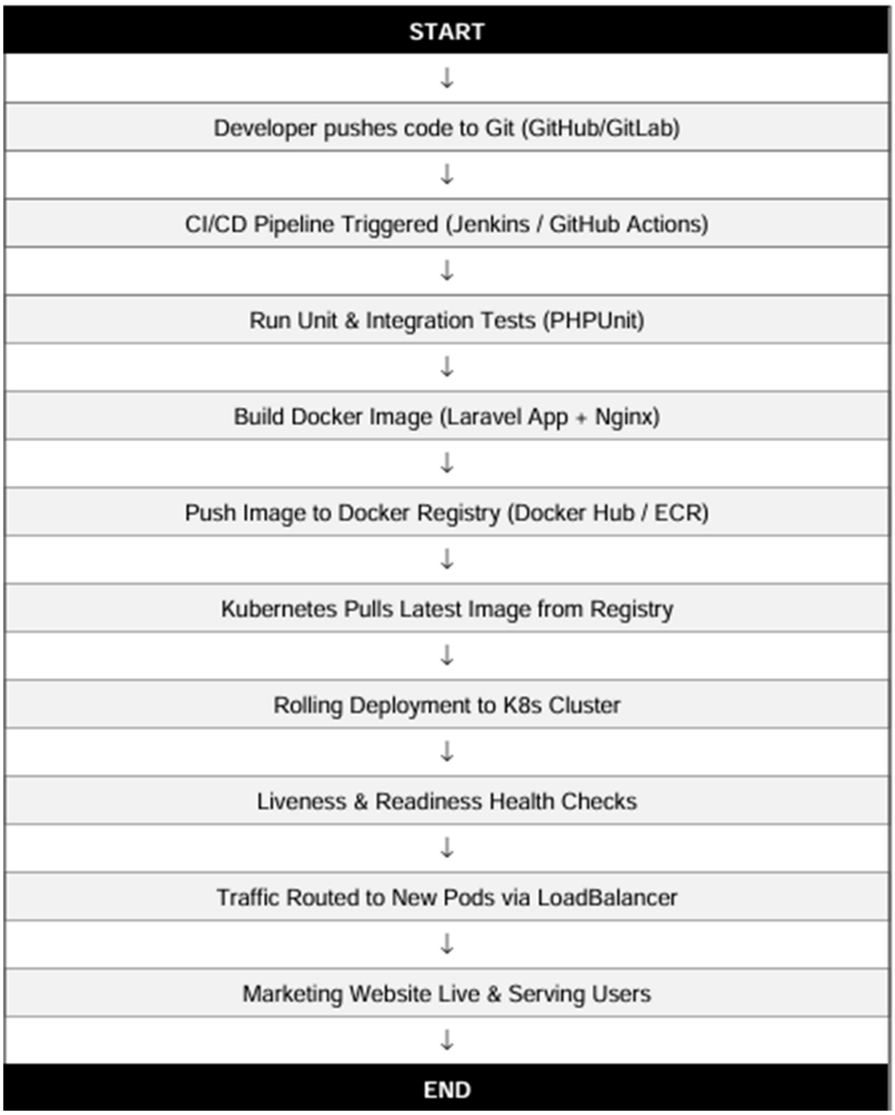
• Orchestration Layer (Kubernetes) → Pod management, scaling, load balancing

• Application Layer (Laravel) → PHP business logic, routing, Eloquent ORM • Data Layer (MySQL + Redis) → Persistent storage and caching

Dockerfile
Defines how the Docker image is built



2.2. Process Flow



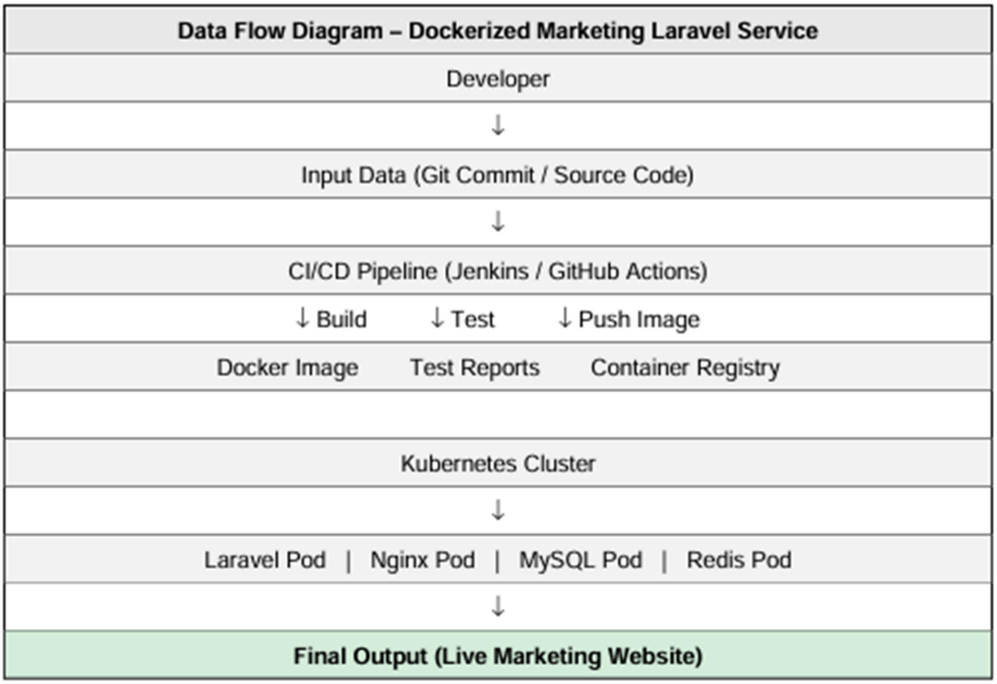
The process starts with a developer pushing code to Git, then the CI/CD pipeline automatically builds the Docker image, runs tests, and pushes to the registry. Kubernetes then performs a rolling deployment ensuring zero-downtime updates of the marketing Laravel application.

2.3. Information Flow

• Input → Git commit (source code, Dockerfile, K8s manifests)

• Intermediate → Docker image, test reports, deployment configs

• Output → Live Dockerized Laravel marketing service on K8s



2.4. Components Design

Components:

• Git Repository

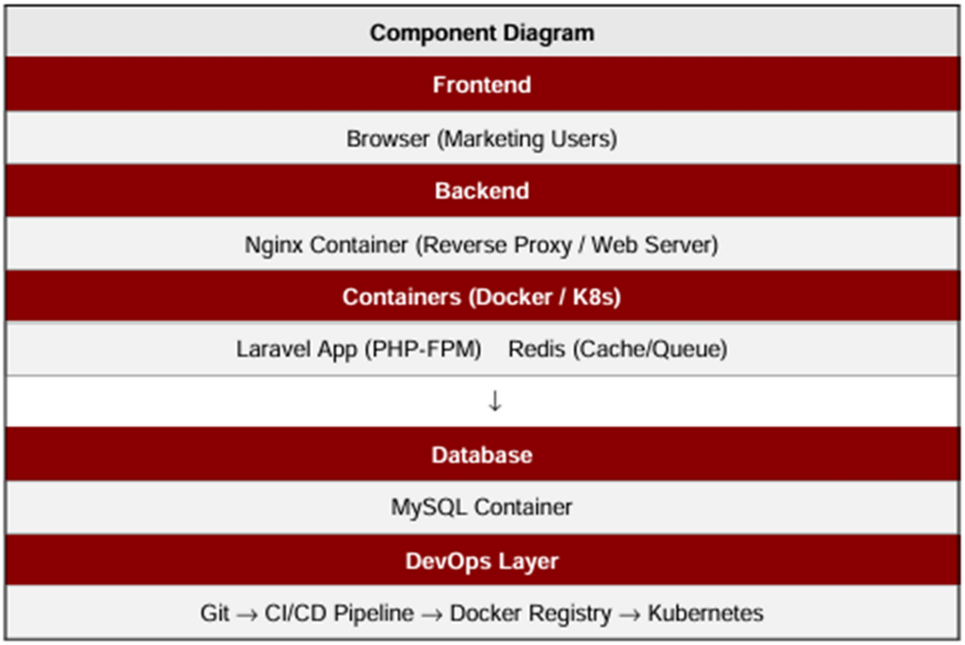
• CI/CD Pipeline (Jenkins / GitHub Actions)

• Docker Engine

• Laravel App Container

• Kubernetes Cluster

• Docker Registry



2.5. Key Design Considerations

The system is designed with the following considerations:

1. Containerization
   - Ensures consistent environment across systems

2. Portability
   - Application can run anywhere using Docker

3. Scalability
   - Can be extended to Kubernetes deployment

4. Isolation
   - Each service runs in its own container

5. Security
   - Dependencies are isolated inside containers

6. Maintainability
   - Easy to update and manage services independently

2.6. API Catalogue

1. Get All Campaigns GET /api/campaigns

2. Create Campaign POST /api/campaigns

Example:
- GET / → Laravel Welcome Page

Future scope includes:
- REST APIs for marketing services
- CRUD operations for campaigns, users, analytics

3. References

□ DevOps Concepts and Best Practices

□ Docker Official Documentation

□ Kubernetes Official Documentation

□ Jenkins CI/CD Documentation

□ GitHub Actions Documentation

□ Laravel PHP Framework Documentation

□ Git Version Control Documentation

□ Nginx Documentation

□ MySQL Documentation

□ Redis Documentation